User Flow & Technical Workflow Specification

Project: Adaptive AI-Tutor & Interactive Assessment Platform | **Team**: Purple Hue (Ayush, Kunal, Abhishek)

---

1. Complete User Flow Architecture

The user flow guides students seamlessly from initial onboarding to interactive assessment, performance analytics, and on-demand AI tutoring.

• Step 1: Student Onboarding

Student opens index page (`/`). Enters Name, 3-digit Roll Number, Email, Year, and Batch. Client validates fields, submits `POST /api/login`, saves attributes to `localStorage`, and logs in.

• Step 2: Subject Dashboard & Creator

Student arrives at `/subjects`. Chooses preset cards (Web Hosting, C Programming, Python) or opens '+ Add Custom Quiz' modal to author custom subjects and flashcard questions.

• Step 3: Interactive Quiz Engine

Student takes quiz at `/quiz`. Questions loaded dynamically via `GET /api/questions/`. Options highlighted in real time with progress bar tracking.

• Step 4: Score Review & Analytics

On submission (`POST /api/submit`), student views `/result` with performance badges (Excellent, Good, Practice), question breakdowns, and historical DB attempts.

• Step 5: Profile Dashboard

Student visits `/profile` to view aggregate performance metrics (total tests, pass rate, average score) and complete test history.

• Step 6: AI-Tutor Side Assistant

At any point, student clicks the compact right-edge arrow tab handle (`■`). Opens right-side chat drawer with live subject context, quick suggestion chips, and chemical/math formula subscript rendering.

User Action	Frontend Screen / Trigger	Backend API / System Action
Submit Login Details	`index.html` -> `app.js`	`POST /api/login` -> Insert user into SQLite
Create Custom Quiz	`subjects.html` -> `subject.js`	`POST /api/quiz/create` -> Insert questions into DB
Take Practice Quiz	`quiz.html` -> `quiz.js`	`GET /api/questions/` -> Query DB
Submit Quiz Answers	`quiz.html` -> `submitBtn`	`POST /api/submit` -> Record score attempt
Ask AI Tutor Question	Right Sidebar Drawer (`■`)	`POST /api/tutor/chat` -> Gemini API / Fallback

2. Technical System Workflow & ADK 2.0 Architecture

The technical workflow integrates Flask REST routes, SQLite database storage, Gemini 2.0 LLM reasoning, and Google ADK 2.0 Graph Workflow nodes.

ADK 2.0 Graph Workflow Node Execution Topology

```
START Event -> parse_submission -> validate_and_sanitize_payload
                                   |-- (Security Gate 1 Fail: XSS/SQLi) --> quarantine_node
                                   |-- (valid & difficulty < 3) ----------> auto_grade_node
                                   +-- (valid & difficulty >= 3) ---------> prompt_guard_node
                                                                               |-- (Security Gate 2 Fai
l) --> quarantine_node
                                                                               +-- (clean payload) ----
-----> llm_review_node -> tutor_verification_node (HITL)
```

- **Phase 1: Flask Web Server Routing (`backend/app.py`)**

  Serves static assets, HTML templates, and REST API endpoints with CORS headers enabled. Auto-initializes SQLite database (`init_db()`) and seeds initial questions if empty.

- **Phase 2: Pre-Ingestion Security Boundary (`validate_and_sanitize_payload`)**

  Validates schema types, escapes script tags to block XSS, and strips SQL injection patterns (`UNION SELECT`, `; --`). Diverts malicious payloads to `quarantine_node`.

- **Phase 3: Pre-LLM Prompt Guard Boundary (`prompt_guard_node`)**

  Scans text for prompt injection and jailbreaks (`ignore system instructions`, `DAN mode`). Short-circuits malicious inputs to `quarantine_node` before invoking Gemini.

- **Phase 4: Adaptive Dual-Path Evaluation**

  - **Fast Path (< 3)**: Pure Python deterministic auto-grading.
  - **AI Path (≥ 3)**: Gemini 2.0 essay evaluation on 100-point rubric + `RequestInput` Human-in-the-Loop tutor approval.

- **Phase 5: AI Tutor Right-Edge Drawer (`POST /api/tutor/chat`)**

  Receives student message and active subject context. Executes `make_request(prompt, model='gemini-2.0-flash')` with 10s deadline. Formats output with HTML subscripts (H■O, CO■).

- **Phase 6: Quota Resilience & Fallback Engine**

  If Google AI Studio free tier rate limit (`429 RESOURCE_EXHAUSTED`) is hit, `backend/ai_service.py` returns instant, structured educational study notes.

API Endpoint	Method	Payload / Parameters	Response Output
`/api/login`	POST	`{ studentName, rollNumber, email, year, batch }`	`{ status: 'success', user }`
`/api/questions/`	GET	Subject path parameter	`{ status: 'success', questions }`
`/api/quiz/create`	POST	`{ subject, description, questions }`	`{ status: 'success' }`
`/api/submit`	POST	`{ studentName, rollNumber, subject, score, total }`	`{ status: 'success', result_id }`
`/api/results/`	GET	Roll number parameter	`{ status: 'success', history }`

`/api/user/`	GET	Roll number parameter	`{ status: 'success', user, metrics }`
`/api/tutor/chat`	POST	`{ message, subject, student_name }`	`{ status: 'success', response }`
